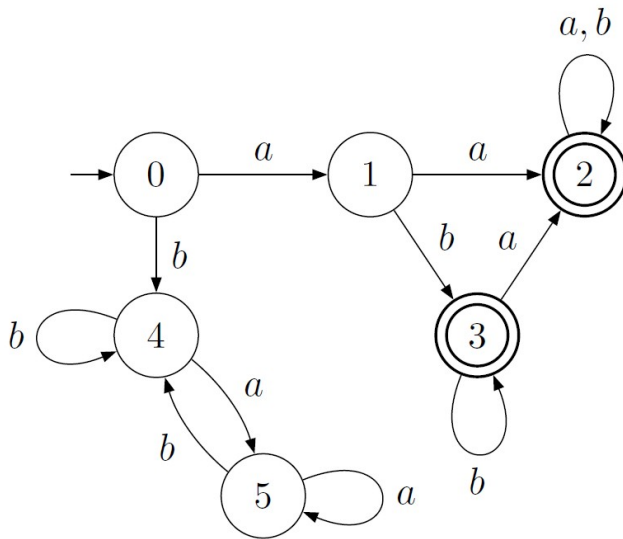




G52LAC (COMP2040)

Languages and Computation

Lecture 6: Minimisation of Finite Automata & Proving Languages not to be Regular

Tomas Maul
School of Computer Science
University of Nottingham Malaysia Campus

Outline

- [5 min] Quick recap.
- Minimisation of Finite Automata.
- Proving Languages not to be Regular.

Outline

- [5 min] Quick recap.
- Minimisation of Finite Automata.
- Proving Languages not to be Regular.

Regular-Language Facts

- For any regular language, there is a **unique minimal** DFA.
- This minimal DFA can be computed.
- REs can be converted into NFAs, and NFAs can be converted into DFAs.
- Two regular languages can be compared for equality by converting them into minimal DFAs.

Equivalence of States

- To minimise a DFA, we need to know which states are equivalent.
- Definition: For any DFA $(Q, \Sigma, \delta, q_0, F)$, $p, q \in Q$ are **equivalent** states iff:

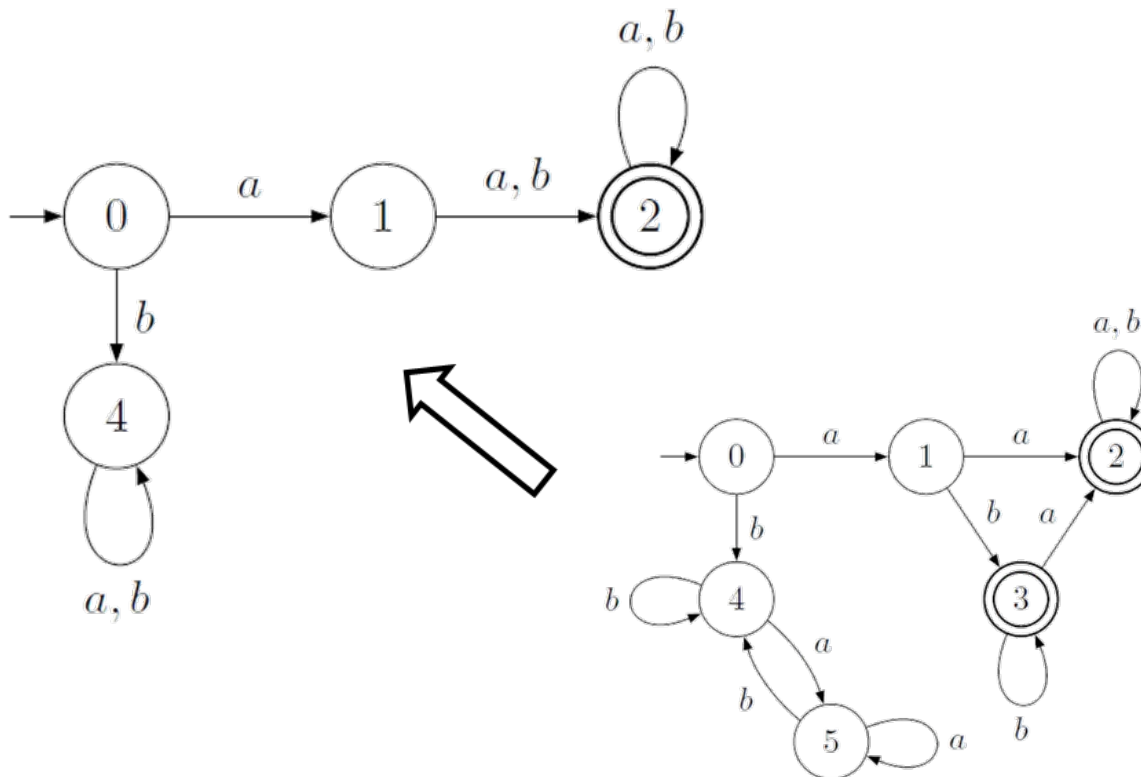
Intuitively: if a word is accepted from one state, it must be accepted from the other, and vice versa.

$$\forall w \in \Sigma^* . \quad \hat{\delta}(p, w) \in F \quad \Leftrightarrow \quad \hat{\delta}(q, w) \in F$$

- Important Note: $\hat{\delta}(p, w)$ and $\hat{\delta}(q, w)$ do **not** have to be the **same** state.
- Definition: If two states are not equivalent, then they are **distinguishable**.

Equivalence of States

Intuition: fuse equivalent states → minimization



- To minimise a DFA, we need to know which states are equivalent.
- Definition: For any DFA $(Q, \Sigma, \delta, q_0, F)$, $p, q \in Q$ are **equivalent** states iff:

$$\forall w \in \Sigma^* . \quad \hat{\delta}(p, w) \in F \quad \Leftrightarrow \quad \hat{\delta}(q, w) \in F$$

- Important Note: $\hat{\delta}(p, w)$ and $\hat{\delta}(q, w)$ do **not** have to be the **same** state.
- Definition: If two states are not equivalent, then they are **distinguishable**.

The Table-Filling Algorithm

- Assume a DFA $(Q, \Sigma, \delta, q_0, F)$
- The algorithm is defined inductively
 - Basis:

$$\forall p, q \in Q.$$

$$(p \in F \wedge q \notin F) \vee (p \notin F \wedge q \in F) \Rightarrow \textit{distinguishable}(p, q)$$

- Induction:

$$\forall p, q \in Q, x \in \Sigma.$$

$$\textit{distinguishable}(\delta(p, x), \delta(q, x)) \Rightarrow \textit{distinguishable}(p, q)$$

- Theorem: If two states are not distinguishable by the Table-Filling Algorithm, then they are equivalent.

Notation

We write

$$\frac{(p, q)}{\begin{array}{l} (p_1, q_1) \\ (p_2, q_2) \\ \dots \\ (p_n, q_n) \end{array}}$$

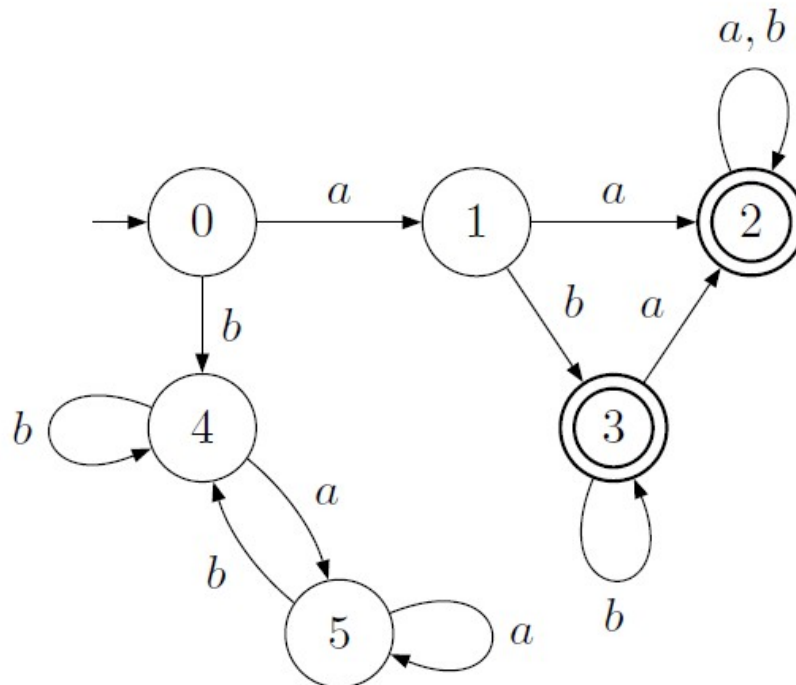
to mean

$$\text{distinguishable}(p, q) \Rightarrow \forall i, 1 \leq i \leq n. \text{distinguishable}(p_i, q_i)$$

Example (1)

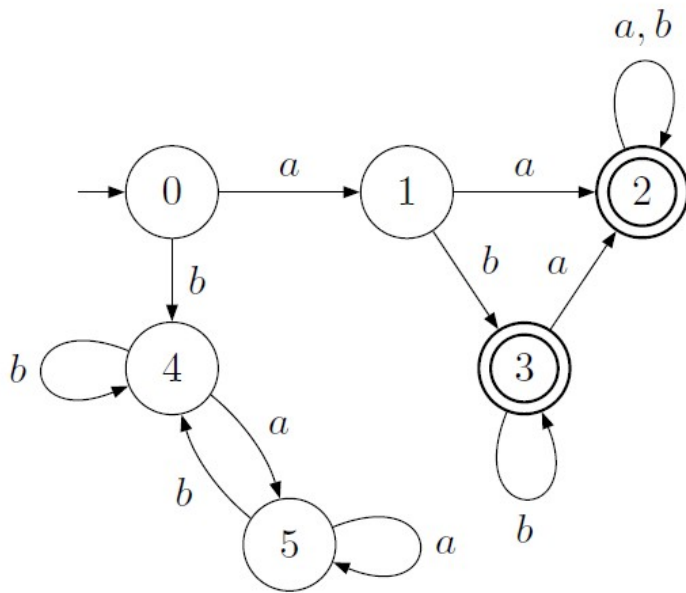
For a DFA $(Q, \Sigma, \delta, q_0, F)$, $p, q \in Q$ are *equivalent* states if and only if, for all $w \in \Sigma^*$, $\hat{\delta}(p, w) \in F \Leftrightarrow \hat{\delta}(q, w) \in F$. If two states are *not* equivalent, then they are *distinguishable*.

Consider the following DFA, where $\Sigma = \{a, b\}$, $Q = \{0, 1, 2, 3, 4, 5\}$, $F = \{2, 3\}$:



Example (2)

The states 1 and 2 are distinguishable on ϵ since $\hat{\delta}(1, \epsilon) = 1 \notin F$ while $\hat{\delta}(2, \epsilon) = 2 \in F$. Similarly, 0 and 1 are distinguishable on e.g. b since $\hat{\delta}(0, b) = 4 \notin F$ while $\hat{\delta}(1, b) = 3 \in F$. On the other hand, in this case, we can easily see that 4 and 5 are not distinguishable on any word since it is not possible to reach any accepting state from either 4 or 5.



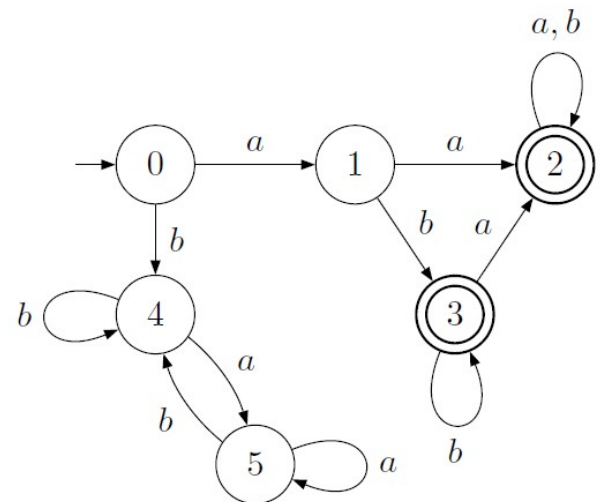
The *Table-Filling Algorithm* recursively constructs the set of *distinguishable* pairs of states for a DFA. When all *distinguishable* state pairs have been identified, any remaining pairs of states must be *equivalent*. Such states can be *merged*, thereby **minimizing** the automaton.

Example (3)

Table Filling ...

First construct a table over all pairs of distinct states. That is, we do not consider pairs $(p \in Q, p \in Q)$ since a state obviously cannot be distinguishable from itself. An easy way of doing constructing the table is to order the states (e.g. numerically or alphabetically), and then list all states except the last one in *ascending* order along the top, and all states except the first one in *descending* order down the left-hand side of the table. The resulting table for our 6-state DFA looks like this:

	0	1	2	3	4
5					
4					
3					
2					
1					



Example (4)

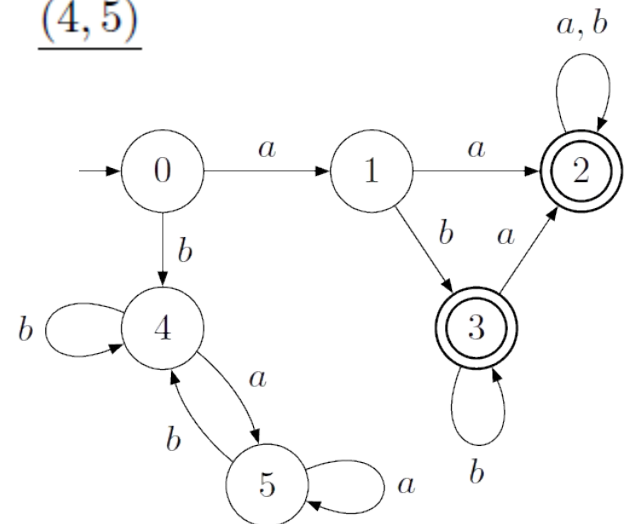
Table Filling ...

Then mark the state pairs that are distinguishable according to the basis of the table-filling algorithm, i.e. the pairs where one state is accepting, and one is not. The accepting states of our DFA are 2 and 3. Thus the state pairs (0, 2), (0, 3), (1, 2), (1, 3), (2, 4), (2, 5), (3, 4), and (3, 5) have to be marked. List all remaining state pairs to the right: these are the potentially equivalent states that we now have to investigate further:

	0	1	2	3	4
5			x	x	
4			x	x	
3	x	x			
2	x	x			
1					

(0, 1) (0, 4) (0, 5) (1, 4)

(1, 5) (2, 3) (4, 5)



Example (5)

Table Filling ...

Recall that the induction step of the table-filling algorithm says that for $p, q, r, s \in Q$ and $a \in \Sigma$, if $(r, s) = (\delta(p, a), \delta(q, a))$ is distinguishable (on some word w), then (p, q) is (on the word aw). If, during systematic investigation of all state combinations, we, from a state pair (p, q) on some input symbol a , reach a state pair (r, s) for which it is not yet known whether it is distinguishable or not, we record (p, q) under the heading for (r, s) . If it later becomes clear that (r, s) is distinguishable, that means that (p, q) also is distinguishable, and recording this implication allows us to carry out the deferred marking at that point.

We write

(p, q)

(p_1, q_1)

(p_2, q_2)

$\dots$

(p_n, q_n)

to mean

$\text{distinguishable}(p, q) \Rightarrow \forall i, 1 \leq i \leq n. \text{distinguishable}(p_i, q_i)$

Example (6)

Table Filling ...

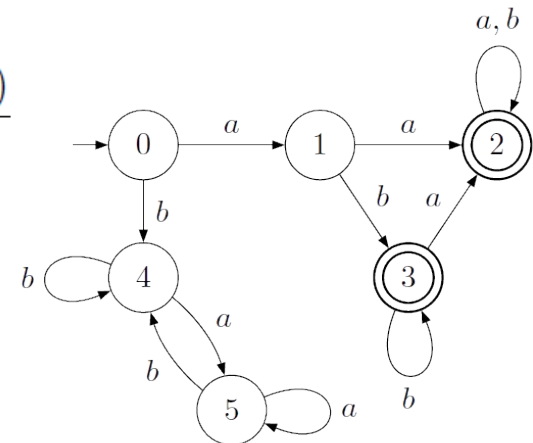
Investigate all potentially equivalent state pairs on all input symbols (unless we find that a pair is distinguishable, which means we can stop):

- $(0, 1): (\delta(0, a), \delta(1, a)) = (1, 2)$ Distinguishable! Mark in table.
 $(0, 4): (\delta(0, a), \delta(4, a)) = (1, 5)$ Unknown as yet. Add **(0,4)** under $(1, 5)$.
 $(\delta(0, b), \delta(4, b)) = (4, 4)$ Same state, no info.

Our table now looks as follows (we strike a line across the pairs we have considered):

	0	1	2	3	4
5			X	X	
4			X	X	
3	X	X			
2	X	X			
1	X				

<u>$(0, 1)$</u>	<u>$(0, 4)$</u>	<u>$(0, 5)$</u>	<u>$(1, 4)$</u>
<u>$(1, 5)$</u>	<u>$(2, 3)$</u>	<u>$(4, 5)$</u>	
<u>$(0, 4)$</u>			



Example (7)

Table Filling ...

We continue:

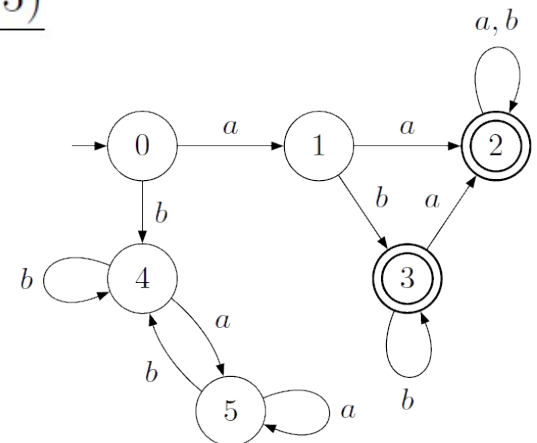
- (0, 5): $(\delta(0, a), \delta(5, a)) = (1, 5)$ Unknown as yet. Add (0, 5) under (1, 5).
 $(\delta(0, b), \delta(5, b)) = (4, 4)$ Same state, no info.
 (1, 4): $(\delta(1, a), \delta(4, a)) = (2, 5)$ Distinguishable! Mark in table.

Table:

	0	1	2	3	4
5			x	x	
4		x	x	x	
3	x	x			
2	x	x			
1	x				

~~(0, 1)~~ ~~(0, 4)~~ ~~(0, 5)~~ ~~(1, 4)~~

(1, 5) (2, 3) (4, 5)
(0, 4)
(0, 5)



Example (8)

Table Filling ...

Now we have come to the state pair $(1, 5)$. If we can determine that $(1, 5)$ is a distinguishable pair, then we also know that the pairs $(0, 4)$ and $(0, 5)$ are distinguishable:

$(1, 5): (\delta(1, a), \delta(5, a)) = (2, 5)$ Distinguishable!

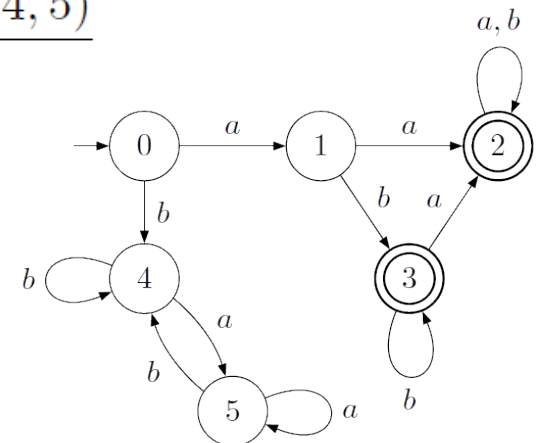
Thus we should mark $(1, 5)$ along with $(0, 4)$ and $(0, 5)$:

	0	1	2	3	4
5	X	X	X	X	
4	X	X	X	X	
3	X	X			
2	X	X			
1	X				

~~$(0, 1)$~~ ~~$(0, 4)$~~ ~~$(0, 5)$~~ ~~$(1, 4)$~~

~~$(1, 5)$~~ $(2, 3)$ $(4, 5)$

~~$(0, 4)$~~
 ~~$(0, 5)$~~

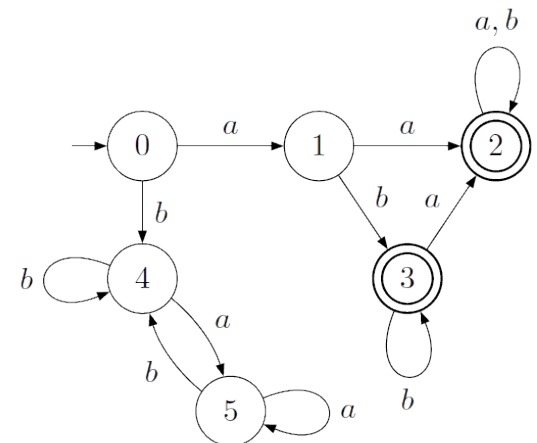


Example (9)

Table Filling ...

It remains to check the pairs (2, 3) and (4, 5):

(2, 3):	$(\delta(2, a), \delta(3, a)) = (2, 2)$	Same state, no info.
	$(\delta(2, b), \delta(3, b)) = (2, 3)$	No point in adding (2, 3) below (2, 3).
(4, 5):	$(\delta(4, a), \delta(5, a)) = (5, 5)$	Same state, no info.
	$(\delta(4, b), \delta(5, b)) = (4, 4)$	Same state, no info.

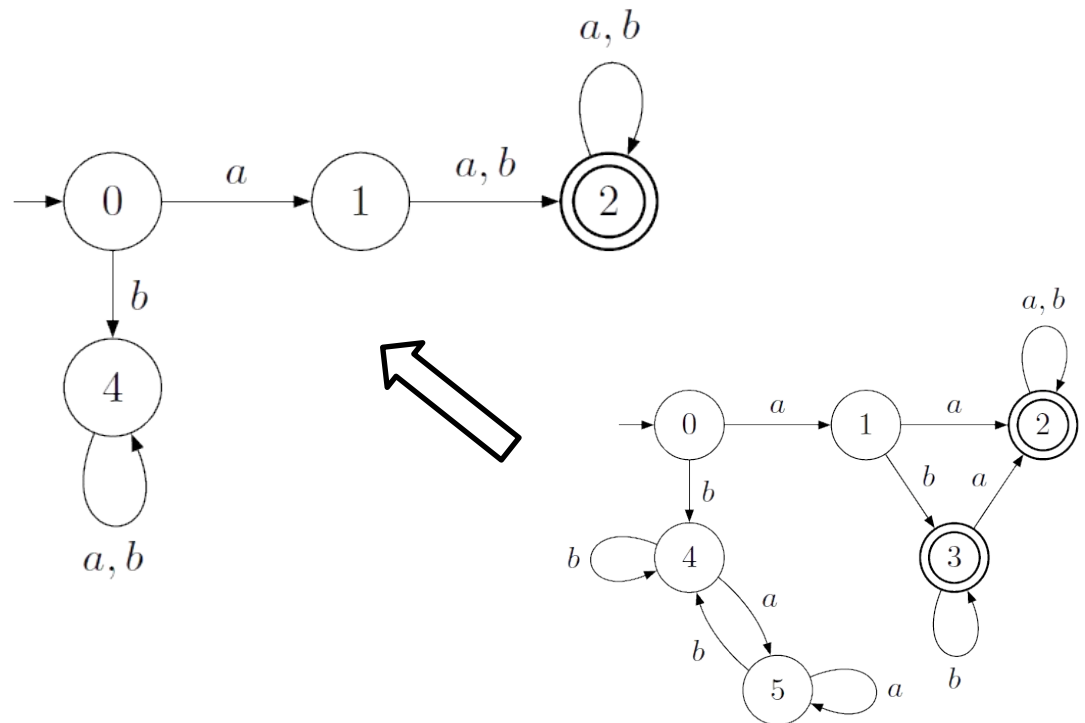


Example (10)

Table Filling ...

We have now systematically checked all potentially equivalent state pairs. Two pairs remain unmarked, i.e. we have not been able to show that they are distinguishable: (2,3) and (4,5). We can therefore conclude that these states are pairwise equivalent: $2 \equiv 3$ and $4 \equiv 5$. We thus proceed to merge these states by, informally, placing them “on top” of each other and “dragging along” the edges. The result is the following minimal DFA (where the merged states have been given the names 2 and 4):

	0	1	2	3	4
5	X	X	X	X	
4	X	X	X	X	
3	X	X			
2	X	X			
1	X				



Recommended Reading

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 155–165
- G52MAL Lecture Notes, pages 25–29

Outline

- [5 min] Quick recap.
- Minimisation of Finite Automata.
- Proving Languages not to be Regular.

Regular Languages

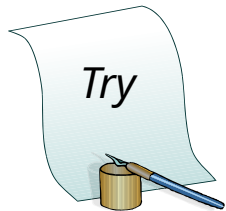
- The Regular Languages are those that can be recognised by **finite** automata.
- Such machines have a finite number states (i.e. finite memory).
- But many languages are not regular.

Example

Regular languages are languages that can be recognized by a computer with finite (i.e. fixed) memory. Such a computer corresponds to a DFA. However, there are many languages that cannot be recognized using only finite memory. A simple example is the language

$$L = \{0^n 1^n \mid n \in \mathbb{N}\}$$

i.e. the language of words that start with a number of 0s followed by the **same** number of 1s. Note that this is different from $L(0^*1^*)$, which is the language of words of a sequences of 0s followed by a sequence of 1s but not necessarily of the same length. (We know this to be regular because it is given by a regular expression.)

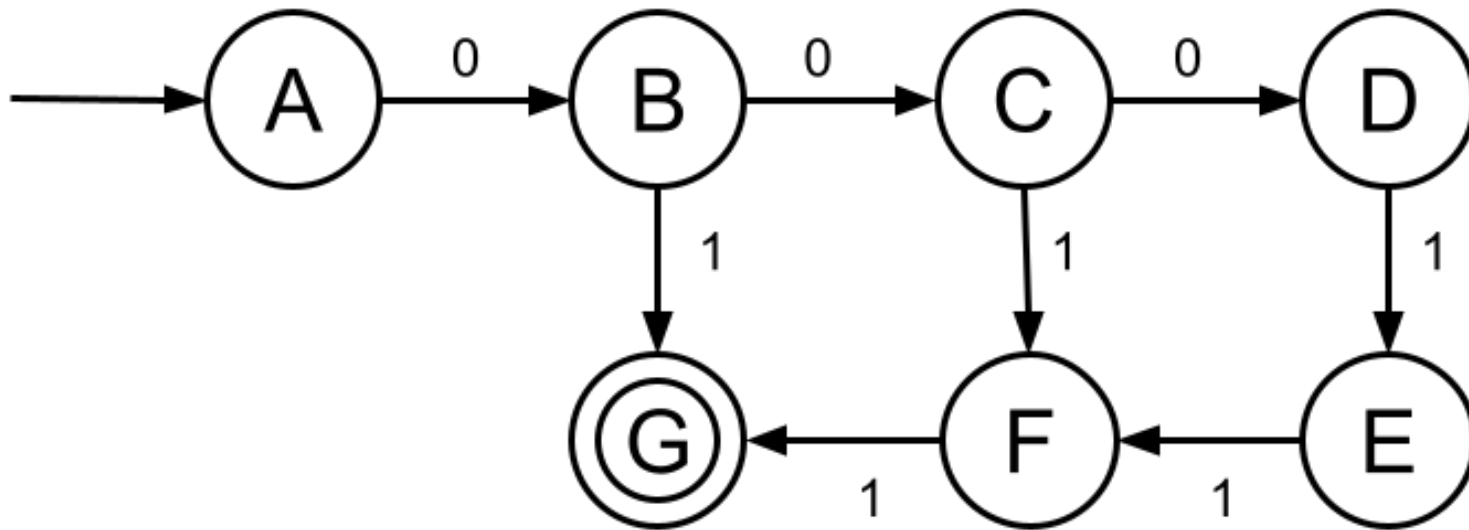


A DFA for $L = \{0^n 1^n \mid n \in \{1,2,3\}\}$

But what about when n belongs to the set of natural numbers?

Example

- A simple NFA.
- You can convert this into a complete DFA by adding a dead state and corresponding transitions.



The Chomsky Hierarchy

All languages

Recursively Enumerable Languages (Type 0)

Turing Machines

Recursive/Decidable Languages

Total Turing Machines / Deciders

Context-Sensitive Languages (Type 1)

Linear-Bounded Turing Machines

Context-Free Languages (Type 2)

Pushdown Automata

Regular Languages (Type 3)

Finite Automata

Proving Languages not to be Regular

Note that when we mention (regular or non-regular) languages in this context, we mean **non-finite** (regular or non-regular) languages.

- How do we **prove** a language is not regular?
- One technique: Using **The Pumping Lemma**
- Basic idea: Exploit the fact that, for any Regular Language, **sufficiently long words are repetitive**.

Rough intuition: a non-finite regular language must have “some pumping somewhere” or must have a loop somewhere in its corresponding finite automaton.

Basic argument:

language is regular $\rightarrow$ language has pumping property

not(language has pumping property) $\rightarrow$ **not**(language is regular)

The Pumping Lemma for Regular Languages (1)

Pumping Lemma

Theorem Given a regular language L , there is a number $n \in \mathbb{N}$ such that **all** words $w \in L$ that have length larger or equal to n ($|w| \geq n$) can be split into three words

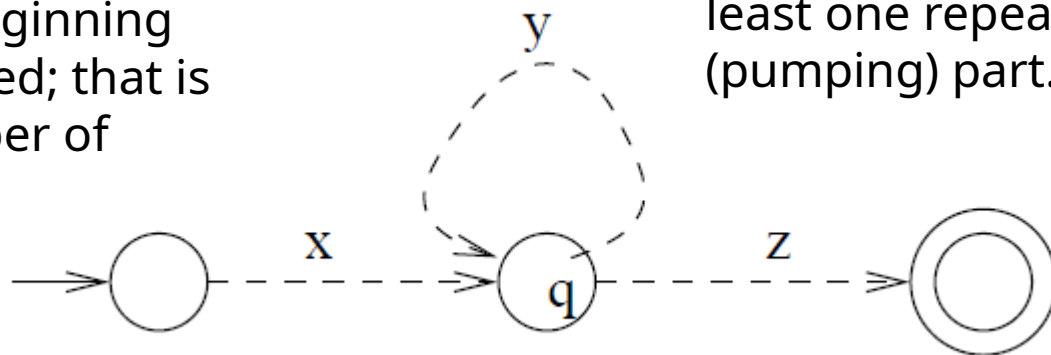
$w = xyz$ s.t.

- $y \neq \varepsilon$
- $|xy| \leq n$
- $\forall k \in \mathbb{N}. xy^kz \in L$

Pay careful attention to “for all” and “there exists”.

Automata book: “That is, we can always find a nonempty string y not too far from the beginning of w that can be pumped; that is, repeating y any number of times ...”

All words of length $\geq n$ can be shown to have at least one repeating (pumping) part.



The Pumping Lemma for Regular Languages (2)

Example

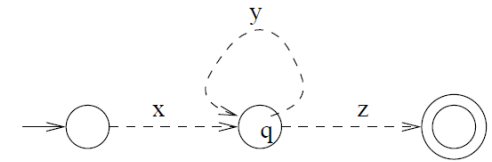
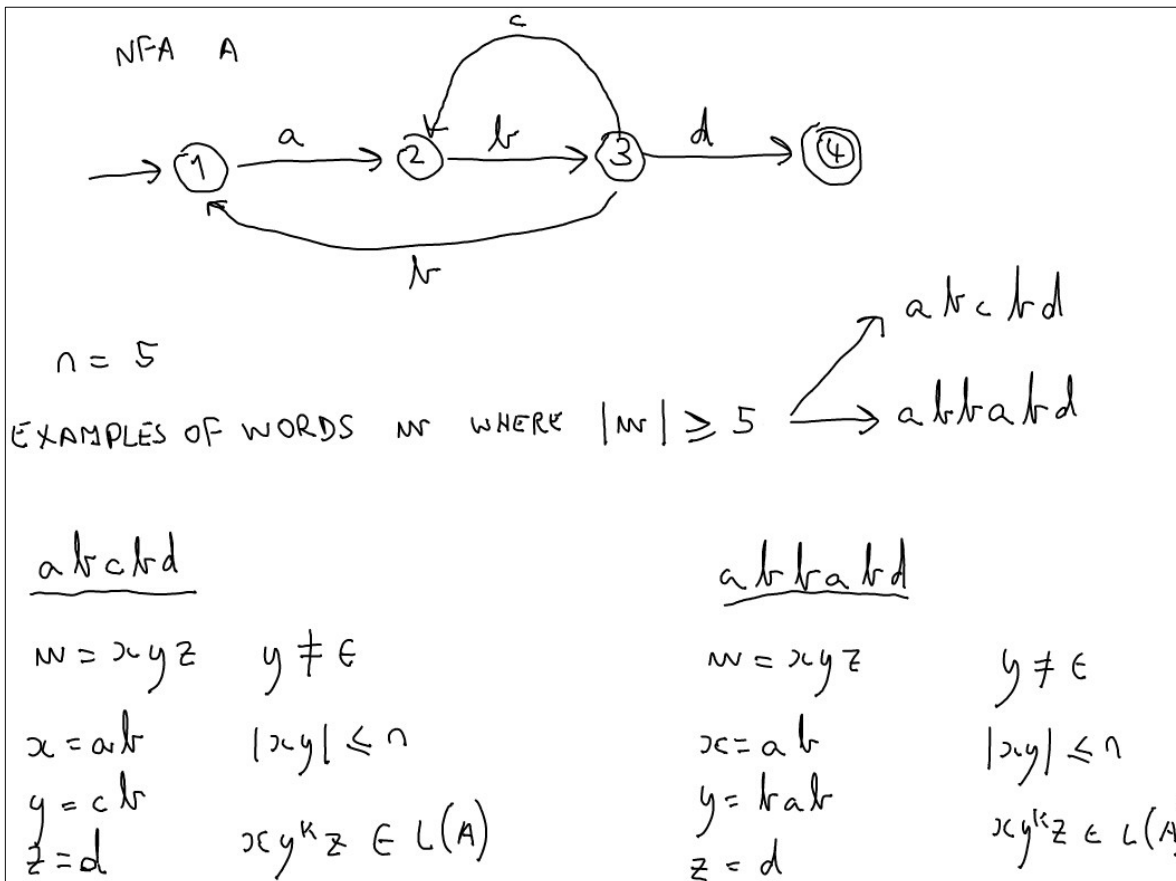
Pumping Lemma

Assuming we have a finite automaton: n must be at least as large as the shortest word with at least one cycle.

Theorem Given a regular language L , there is a number $n \in \mathbb{N}$ such that all words $w \in L$ that have length larger or equal to n ($|w| \geq n$) can be split into three words $w = xyz$ s.t.

- $y \neq \epsilon$
- $|xy| \leq n$
- $\forall k \in \mathbb{N}. xy^kz \in L$

Pay careful attention to "for all" and "there exists".



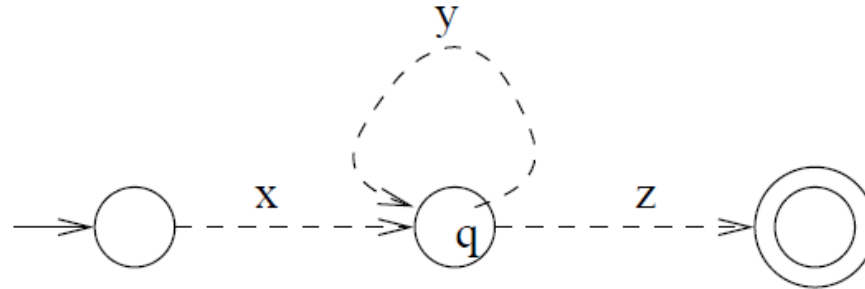
Note: a "cycle" means we transition from a state, through one or more other states, back to the original state, e.g.: 3-2-3 or 3-1-2-3.

You can't find a word, from the language under consideration, larger than n that doesn't have the pumping property.

The Pumping Lemma for Regular Languages (3)

Proof

Proof: For a regular language L there exists a DFA A s.t. $L = L(A)$. Let us assume that A has got n states. Now if A accepts a word w with $|w| \geq n$ it must have visited a state q twice:



Now, consider what happens if we feed a word of the form xy^iz to the automaton, i.e. s instead of y it contains an arbitrary number of repetitions of y , including the case $i = 0$, i.e. y is just left out. The automaton has to accept all such words and hence $xy^iz \in L$

□

The Pumping Lemma for Regular Languages (4)

Applying the Pumping Lemma 1

Theorem Given a regular language L , then there is a number $n \in \mathbb{N}$ such that all words $w \in L$ that are longer than n ($|w| \geq n$) can be split into three words $w = xyz$ s.t.

- $y \neq \epsilon$
- $|xy| \leq n$
- $\forall k \in \mathbb{N}. xy^kz \in L$

All words larger than n must have the pumping property. If you find at least one exception, then you have proven that the language can't be regular.

Theorem The language $L = \{0^n 1^n \mid n \in \mathbb{N}\}$ is not regular.

Proof: Assume L would be regular. We will show that this leads to contradiction using the pumping lemma.

Now by the pumping lemma there is an n such that we can split each word that is longer than n such that the properties given by the pumping lemma hold. Consider $0^n 1^n \in L$, this is certainly longer than n . We have that $xyz = 0^n 1^n$ and we know that $|xy| \leq n$, hence y can only contain 0s, and since $y \neq \epsilon$ it must contain at least one 0. Now according to the pumping lemma $xy^0z \in L$ but this cannot be the case because it contains at least one 0 less but the same number of 1s as $0^n 1^n$.

Hence, our assumption that L is regular must have been wrong.



Basic argument:

language is regular $\rightarrow$ language has pumping property

not(language has pumping property) $\rightarrow$ **not**(language is regular)

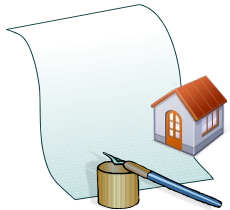
The Pumping Lemma for Regular Languages (6)

Applying the Pumping Lemma 2

Are the following languages regular?

$$\{1^n \mid n \text{ is even}\}$$

$$\{1^n \mid n \text{ is a square}\}$$



Try to solve these before you look at the notes in the MAL pdf.

Recommended Reading

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 127–131
- G52MAL Lecture Notes, pages 29–31

Acknowledgements

- Thorsten Altenkirch.
- Neil Sculthorpe.
- Henrik Nilsson.
- V.P. Kallimani.